Longstaff Schwartz algorithm (how it works):

Note: X is stock price, K is strike price.

1. Use Monte Carlo simulation to generate all of the random future paths. In this case we use the Black Scholes function because it's time continuous and is risk-neutral. Makes it easier for us.

2. Start from T = maturity and compute the option payoffs. max(X - K) for call, max(K-X) for put. Note that continuation value has to = payoff when T = maturity.

3. [INTERMEZZO] - Ignore all paths with zero payoff since they don't contribute to the continuation value calculation.

4. Discount the time T paths payoffs to time T-1 and regress them at time T-1 using a regression function of choice. Regression is used to estimate continuation value.

5. For every path, plug stock price X into the regression function to get the continuation value, and check whether the payoff > continuation value at each path.

6. If true, then the path value = payoff. If not, then the path value = continuation value. (T-1 paths get updated)

7. Go back t-1 increments (eg. 1 month), and repeat steps 3, 4, 5, and 6.

8. Once we reach t = 0, the price of the bermudan option will simply be the weighted average of the discounted paths' values at t = 1.

Extra note:

If we're training and testing on separate data, we simply save the regression coefficients for each path somewhere (policy[i]) after step 4, then plug in the stock prices in the out-of-sample data to get the continuation values.

Why does it work?

Continuation value takes way too long to compute manually.

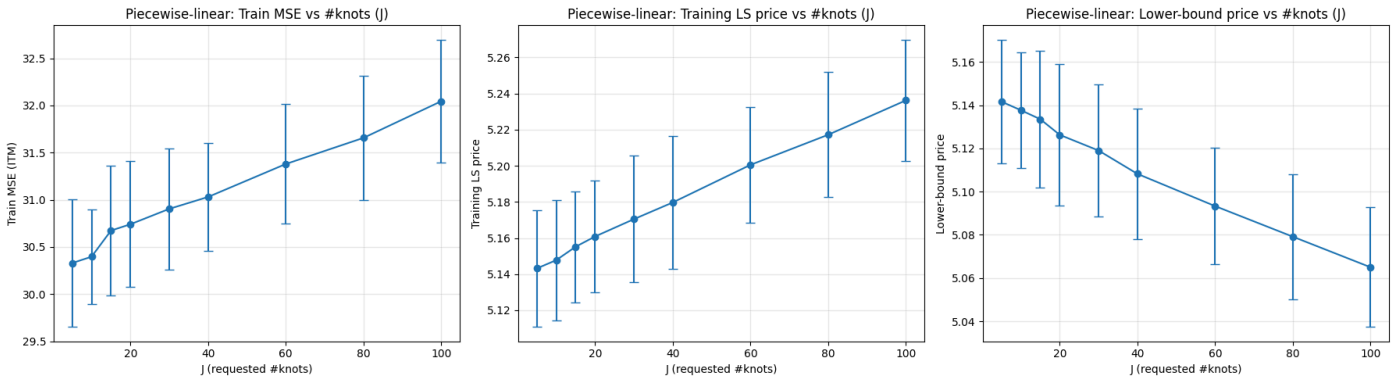
Model Parameter Optimization

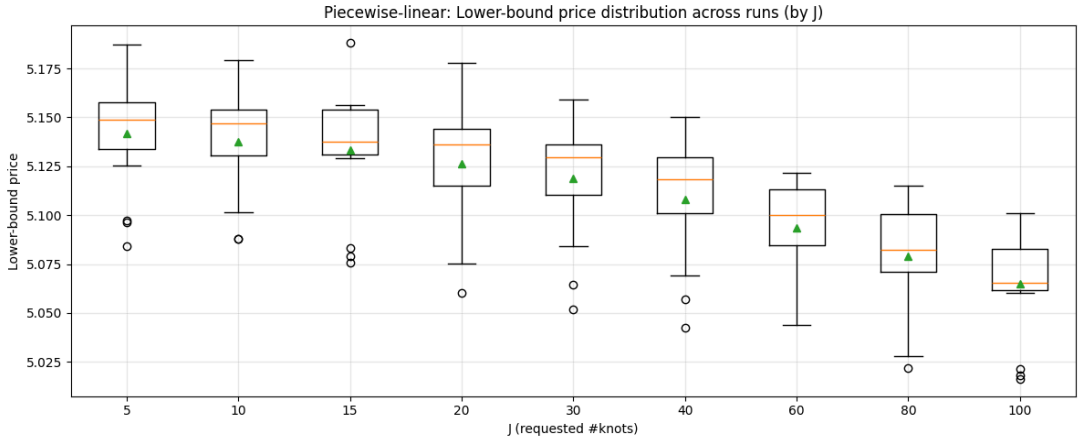
Black Scholes Basis

The Black Scholes Basis regression model does not need any parameters to be optimized.

Piecewise Linear Regression

Piecewise linear regression fits several straight lines to different parts of the data. This allows the model to change slope across ranges of the data, so it can capture different trends more accurately than a single straight line.



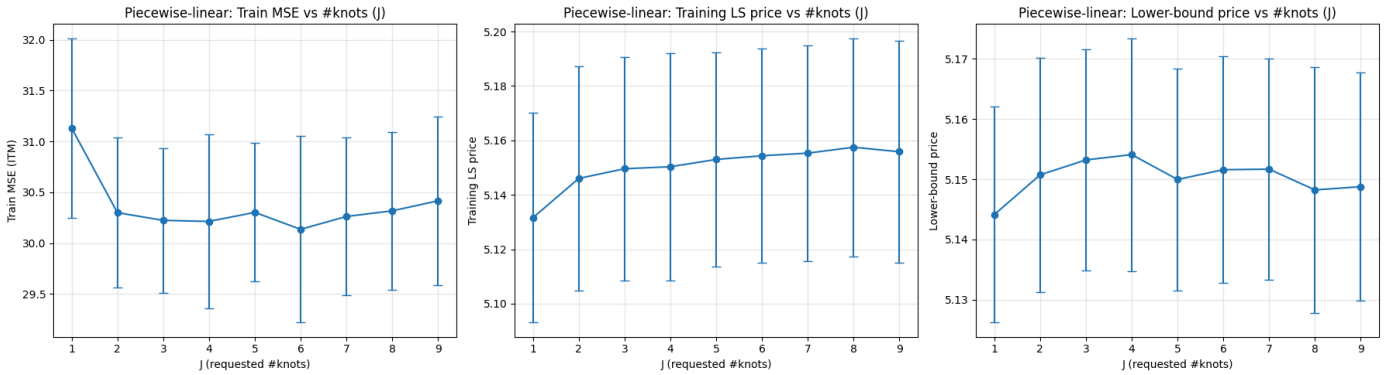


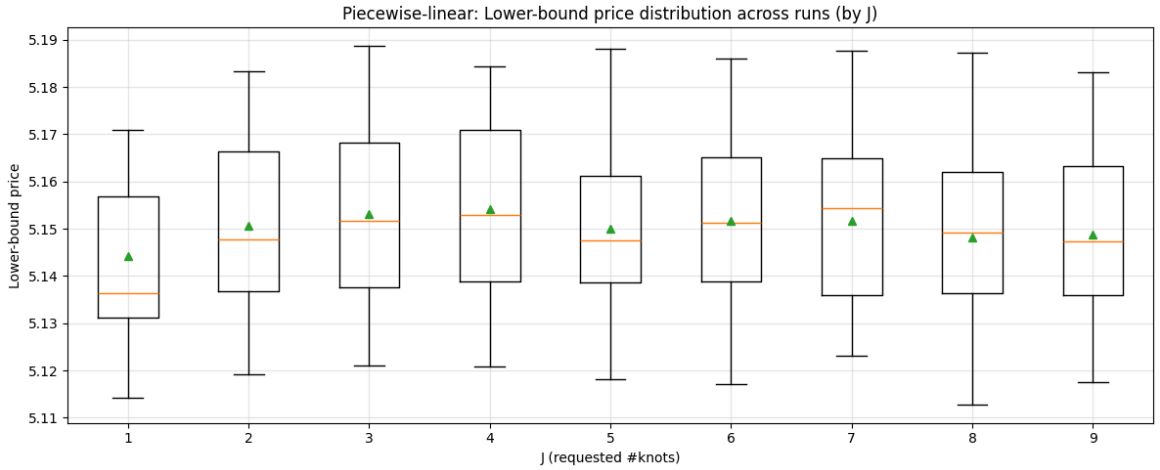
The blue graphs indicate the mean average MSE, training LS price, and out-of-sample lower-bound price with respect to J knots across 15 different monte carlo simulations. The bars represent the prices at +/- 1 standard deviation from the mean. The box plots represent the lower out-of-sample price distribution across 15 different runs. Note that the yellow bar represents the median and the green triangle represents the mean price.

The mean average MSE across 15 runs increases linearly with increasing values of J, suggesting that the model might be overfitting on the data.

The mean average training price also increases linearly, but the average out-of-sample lower bound price over 15 runs decreases linearly with increasing values of J.

Note that the reference price is 5.152, so this suggests that large values of J fail to produce an accurate price for the bermudan option with monthly exercise points. I will further examine smaller values of J to determine the optimal parameter for this regression method.





Here, it would seem that the optimal J that minimizes MSE while also giving an accurate price in relation to the reference (5.152) lies between 3 and 5. I will test this across different monte carlo simulation seeds to see whether this holds true or not.

Extra tests:

```
Knots = 1, LB-price = 5.135543835837713
Knots = 2, LB-price = 5.136531934067064
Knots = 3, LB-price = 5.13527155902514
Knots = 4, LB-price = 5.135344858926694
Knots = 5, LB-price = 5.135882214911101
Knots = 6, LB-price = 5.134018785890226
Knots = 7, LB-price = 5.132508470977913
Knots = 8, LB-price = 5.132586678687011
```

```
Knots = 1, LB-price = 5.1494436690452785
Knots = 2, LB-price = 5.150413302772429
Knots = 3, LB-price = 5.151987164598226
Knots = 4, LB-price = 5.1523810757169635
Knots = 5, LB-price = 5.152227603002257
Knots = 6, LB-price = 5.150408793983471
Knots = 7, LB-price = 5.15021146706499
Knots = 8, LB-price = 5.146692179147201
```

```
Knots = 1, LB-price = 5.147316608713344
Knots = 2, LB-price = 5.150205704204128
Knots = 3, LB-price = 5.151011247761521
Knots = 4, LB-price = 5.150032929479309
Knots = 5, LB-price = 5.1479772511383395
Knots = 6, LB-price = 5.1480121103998275
Knots = 7, LB-price = 5.148214350203516
Knots = 8, LB-price = 5.145507381200454
```

After a couple more runs (including runs that aren't shown in the above pictures), it would seem that the optimal J is **4 knots.** However, it is important to note that at small values of J, the MSE and accuracy to the reference price in out-of-sample prices don't really make a huge difference overall.

In the subsequent accuracy tests, I will use **4 knots as the parameter** for the piecewise-linear regression.

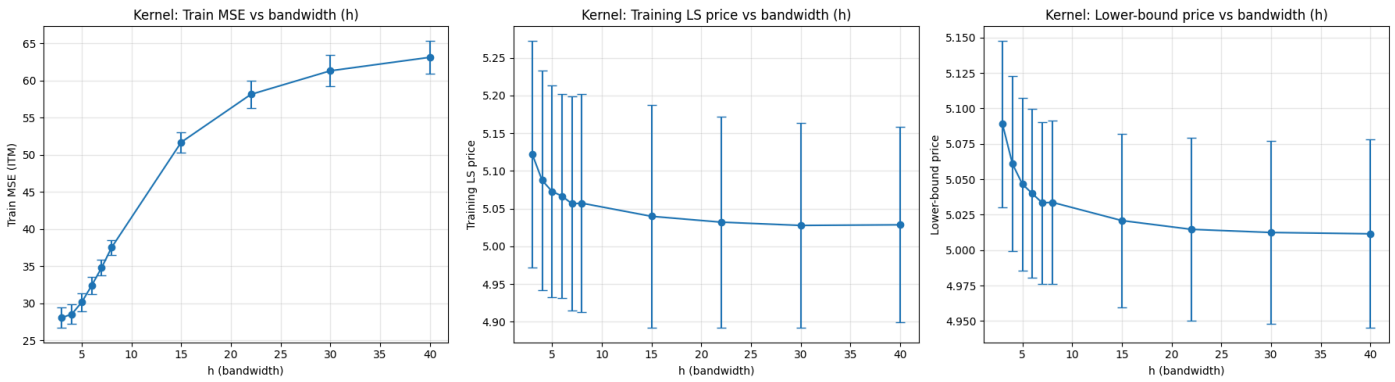
Kernel regression (Gaussian Kernel)

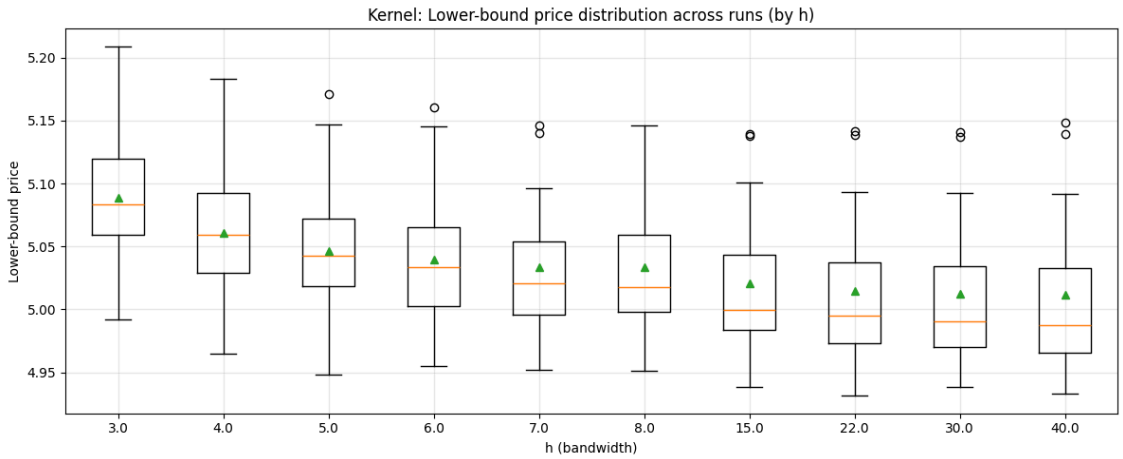
Gaussian kernel (Nadaraya–Watson) regression makes a prediction at a given point by averaging nearby data points. Observations closer to the target point receive more weight, while those farther away receive less weight, according to a Gaussian weighting function.

The bandwidth **(h)** controls how much influence nearby data points have on the prediction. A small **h** means only very close points matter, producing a more detailed but noisier estimate, while a large **h** means more points are averaged, giving a smoother but less responsive estimate.

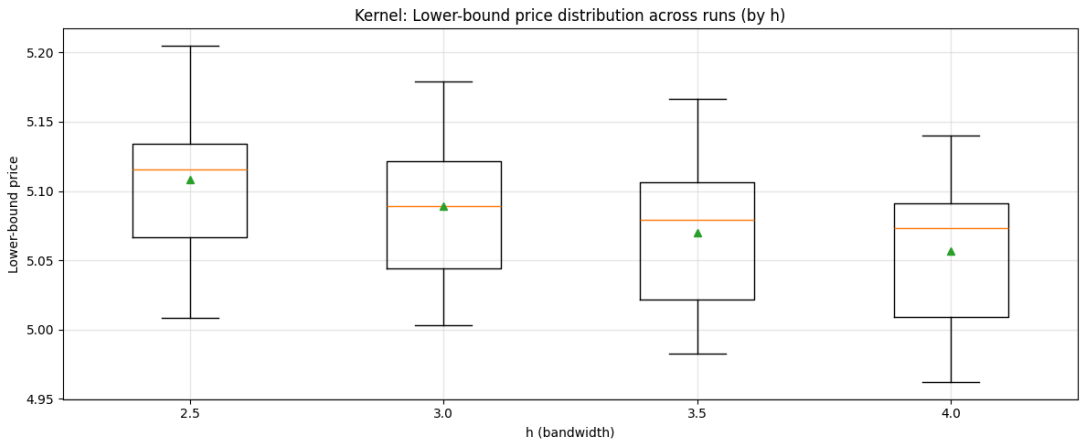
One problem with the kernel regression is that for large values of train / test splits, the model takes an exponentially increasing amount of time. I ran the algorithm with **one single** monte carlo simulation (test uses 15.) Here are my findings:

2000 / 10000 – takes 2.1 seconds total.
5000 / 40000 – takes 14.5 seconds total.
6000 / 48000 – takes 19.5 seconds total.
8000 / 64000 – takes 43 seconds total.
20000 / 10000 – took too long so I didn't measure it. At least 2 minutes.

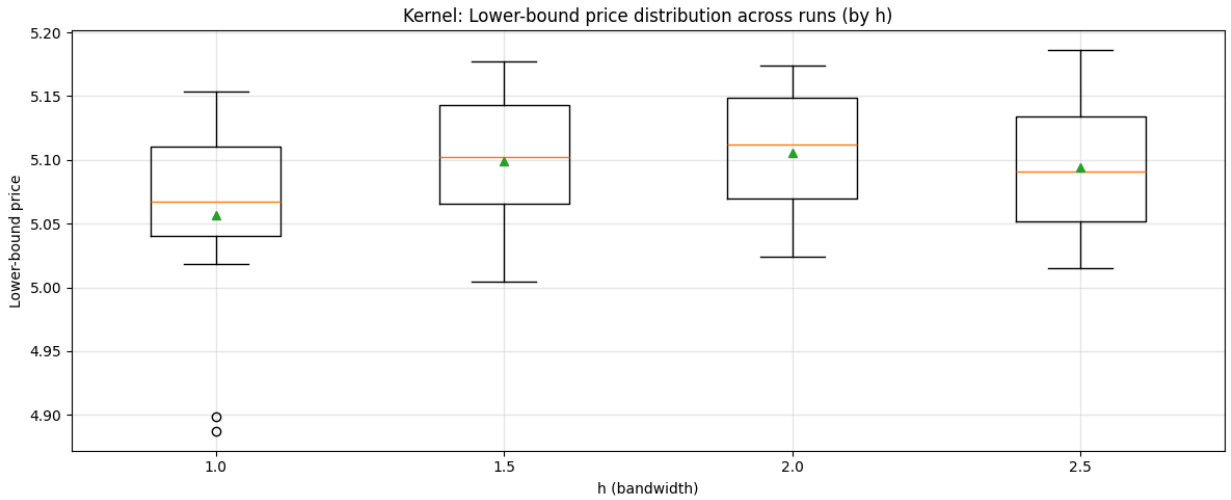




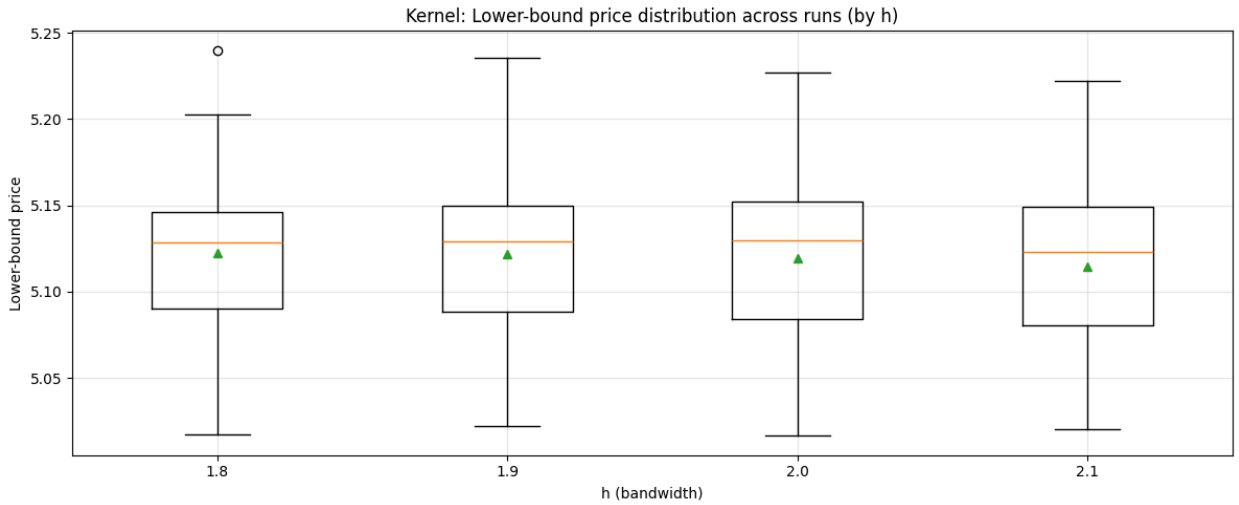
For large values of h, average training MSE skyrockets exponentially, making large values of h unreliable for accurate pricing of the bermudan option. Additionally, the mean out-of-sample price decreases with increasing values of h. This suggests that the **optimal range of h is between 1 - 5.**



I tested this from h = 2.5 to h = 4.0, again the results are very similar – a linearly decreasing lower bound price. (Reference is 5.152).

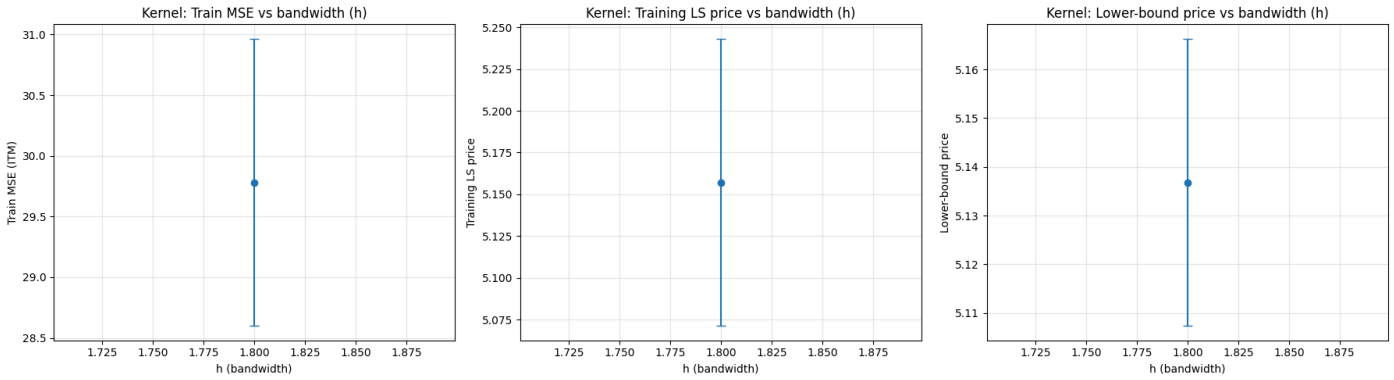


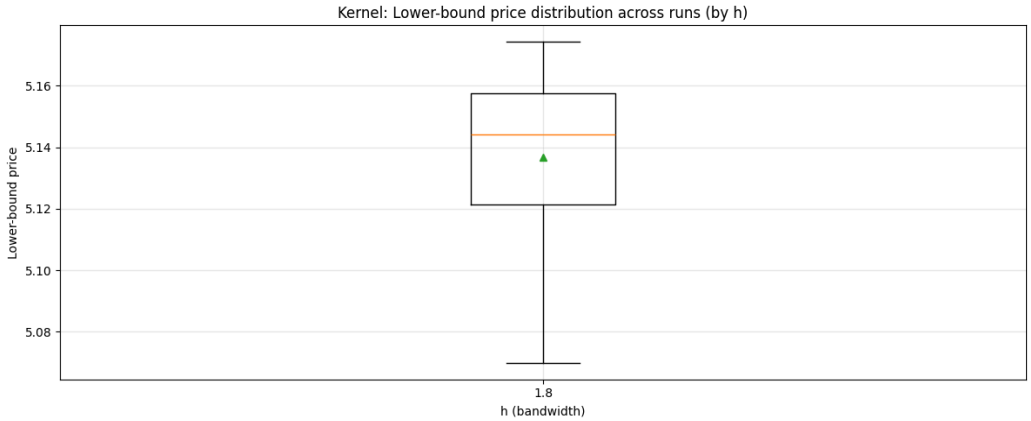
Upon testing from h = 1.0 to h = 2.5, the lower bound price seems to follow a parabolic curve with a maximum value. I will determine this maximum value using further testing and smaller increments of h.



Looking at the lower bound price, it seems that the optimal **h** is **h = 1.8**. I will use this in further testing from now on.

Note that the train/test split used above is 2000 / 10000. It could be a concern that such low values of train / test split in relation to the other models (20000 / 100000) might cause inaccurate pricing. Therefore, I will backtest the kernel regression with **h = 1.8** with a higher train/test split (5000 / 40000) to ensure that the model's pricing is still somewhat accurate, taking MSE into consideration as well.





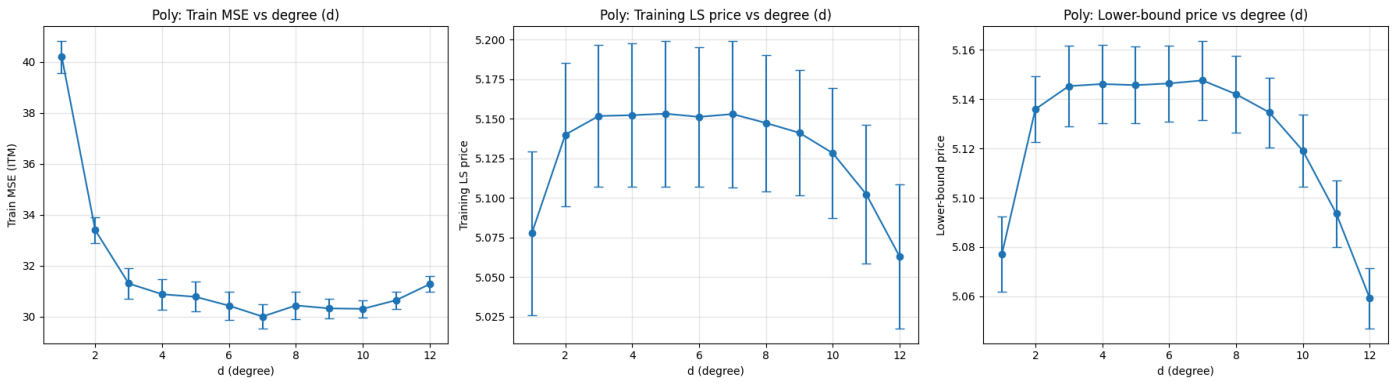
It seems that the model still retains its accuracy with h = 1.8 when subject to higher train / test splits. (5000 / 40000). However, the model took 2 minutes and 28 seconds to execute, making this pricing model **extremely unsuitable in real-world conditions** due to high computational load and slow execution.

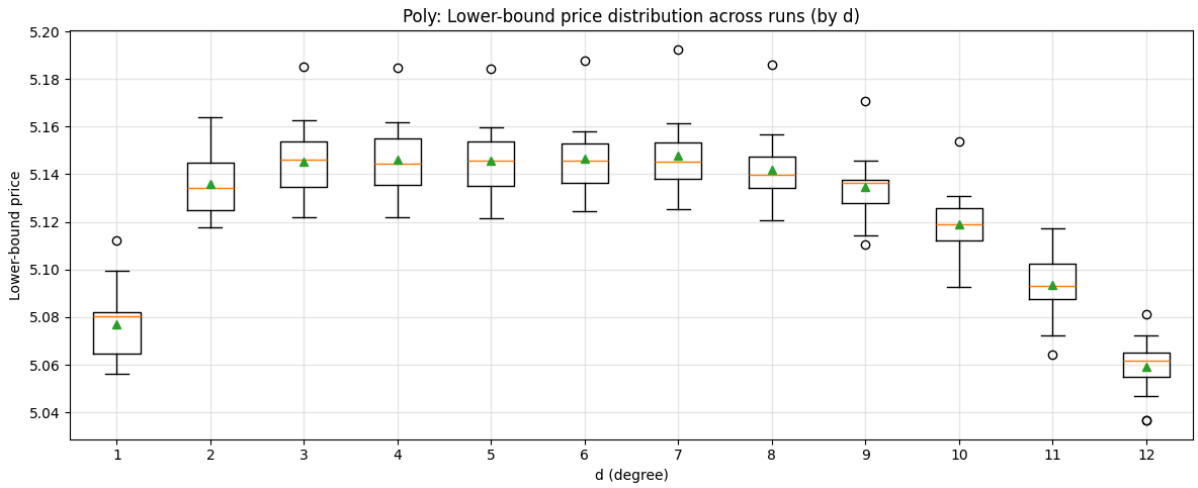
Therefore, the Kernel Regression will not be considered in further accuracy and robustness tests from now on.

Polynomial Regression

Polynomial regression models the relationship between variables by fitting a curved line defined by polynomial terms. This allows the model to capture nonlinear patterns, but higher-degree polynomials can lead to overfitting.

The degree of the polynomial controls model flexibility, with higher degrees increasing variance and the risk of overfitting.





From the data, it seems that the optimal degree lies between 3 and 7. Further runs will be conducted to test this hypothesis.

Extra tests (Selecting the one with the closest reference value):

Test 1: d = 4, avg lower bound = 5.142
Test 2: d = 6, avg lower bound = 5.150
Test 3: d = 6, avg lower bound = 5.150
Test 4: d = 8, avg lower bound = 5.154
Test 5: d = 8, avg lower bound = 5.153
Test 6: d = 5, avg lower bound = 5.152
Test 7: d = 4, avg lower bound = 5.148
Test 8: d = 6, avg lower bound = 5.144
Test 9: d = 4, avg lower bound = 5.149
Test 10: d = 6, avg lower bound = 5.151
Test 10: d = 6, avg lower bound = 5.151

From d = 3 to 8, the differences in prices and MSE are quite small (+/- 0.5 difference for prices +/- 1 difference for MSE at most). Therefore, based on the results, I will choose **polynomial degree = 6** for further accuracy and robustness tests.

Model Comparison Tests

Parameters:
Piecewise Linear Regression: **J = 4 knots**
Polynomial Regression: **degree = 6**

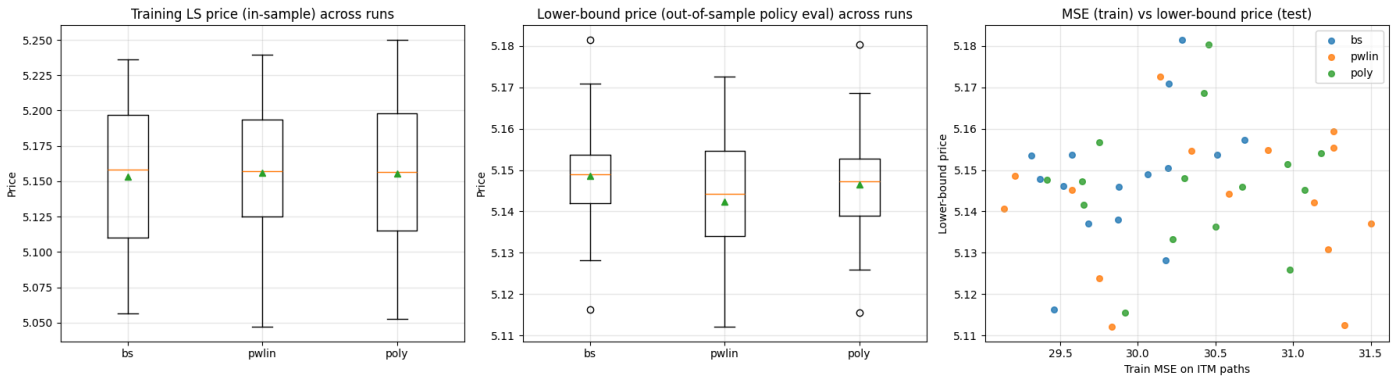
Regression methods tested:

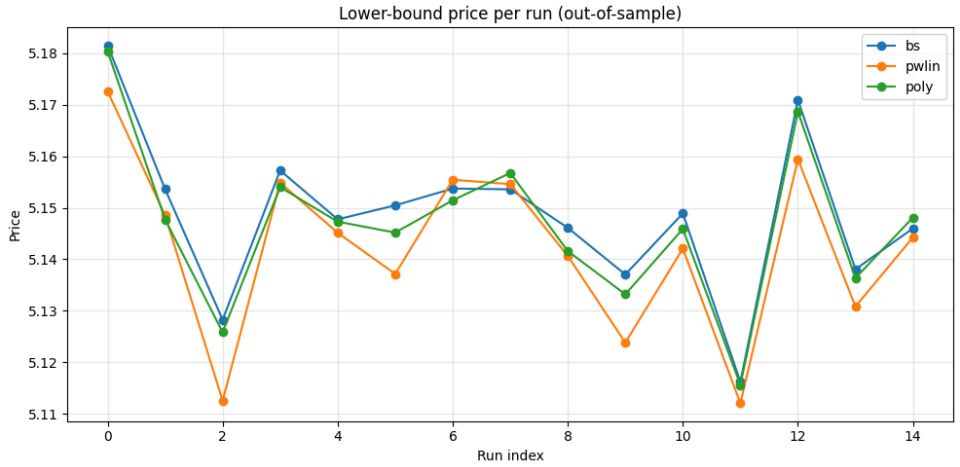
1. Black-Scholes Basis Regression
2. Piecewise Linear Regression
3. Polynomial Regression

Number of runs: 15, 30 (using same seed for the first 15 monte carlo paths)

```
Training LS prices + MSE:
   Black-Scholes basis:          price=5.153473, mse=29.919105
   Piecewise-linear:             price=5.156298, mse=30.474708
   Polynomial regression (deg 6): price=5.155552, mse=30.343199

Lower-bound prices (out-of-sample policy evaluation):
   Black-Scholes basis:          5.148630
   Piecewise-linear:             5.142264
   Polynomial regression (deg 6): 5.146541
```





All three pricing algorithms seem to be quite accurate at predicting the reference Option Price (5.152). MSE seems to be pretty evenly spread out between the algorithms, with all values within 29 and 32 without any massive outliers. This suggests that the pricing models are robust across different runs.

All three regression methods seem to have almost identical prices for each run. In particular, the black-scholes basis and the polynomial regression are almost identical in pricing, with the piecewise linear regression not being too far off.

The results show that when predicting the reference price, the Black-Scholes basis seems to be the most accurate. However, further testing using WRDS Option data is needed.

Testing increasing number of runs (n_runs):

When we increase the number of **independent runs to n = 30**, here are the results:

```
MEAN results across n_runs=30 independent runs
Training LS prices + MSE:
   Black-Scholes basis:           price=5.160958, mse=29.988658
   Piecewise-linear:              price=5.164319, mse=30.460750
   Polynomial regression (deg 6): price=5.163717, mse=30.431895

Lower-bound prices (out-of-sample policy evaluation):
   Black-Scholes basis:           5.146750
   Piecewise-linear:              5.140503
   Polynomial regression (deg 6): 5.144547
```

When we increase the number of **independent runs to n = 50,** here are the results:

```
MEAN results across n_runs=50 independent runs
Training LS prices + MSE:
   Black-Scholes basis:           price=5.157903, mse=30.040981
   Piecewise-linear:              price=5.160473, mse=30.432569
   Polynomial regression (deg 6): price=5.160646, mse=30.492358

Lower-bound prices (out-of-sample policy evaluation):
   Black-Scholes basis:           5.148159
   Piecewise-linear:              5.142556
   Polynomial regression (deg 6): 5.146434
```

When we increase the number of **independent runs to n = 100**, here are the results:

```
MEAN results across n_runs=100 independent runs
Training LS prices + MSE:
   Black-Scholes basis:           price=5.150211, mse=29.999020
   Piecewise-linear:              price=5.152798, mse=30.379362
   Polynomial regression (deg 6): price=5.152006, mse=30.439872

Lower-bound prices (out-of-sample policy evaluation):
   Black-Scholes basis:           5.151899
   Piecewise-linear:              5.147147
   Polynomial regression (deg 6): 5.150023
```

When we increase the number of **independent runs to n = 150**, here are the results:

```
MEAN results across n_runs=150 independent runs
Training LS prices + MSE:
   Black-Scholes basis:           price=5.150498, mse=29.963681
   Piecewise-linear:              price=5.153539, mse=30.331129
   Polynomial regression (deg 6): price=5.152428, mse=30.401875

Lower-bound prices (out-of-sample policy evaluation):
   Black-Scholes basis:           5.151984
   Piecewise-linear:              5.147621
   Polynomial regression (deg 6): 5.150428
```

NOTE: Algorithm takes 46.5 seconds to run here.

The results suggest that **increasing the number of runs** leads to a **more accurate output** (closer to the reference value) across all three regression methods. This is because of the law of large numbers, which says that as you average more independent Monte Carlo estimates, the sample mean converges to the true expected value.

However, for time-saving and real-life usage purposes, I will use n = 30 runs.

Time Granularity Tests

So far all of the tests that have been conducted were using bermudan options with 12 time splits (Jan, Feb, Mar…) to approximate the price of an American Option. We seem to be able to price the American Option accurately within the reference price (5.152). **However, is this robust enough when pricing other 1-year American Options with a different price?**

The option data used in this part is real market data from WRDS:

AAPL American put option count: 20
Date: July 1st, 2024
Strike range: $100 to $230
Time to maturity: 1 year
AAPL Spot: $186.87
Risk-free rate: 5.51% (WRDS OptionMetrics Zero Curve for 1 year maturity)
Implied ATM Volatility* : 23.4193%
12-Month Dividend yield: 0.5018%*

* Implied ATM Allows for more accurate risk-neutral pricing. Option pri
* Dividend yield (q) is calculated by using y = (0.23 + 0.23 + 0.24 + 0.24) / 186.87, then q = ln(1+y)

Regression method: Black-Scholes Basis Regression Model (Most accurate)

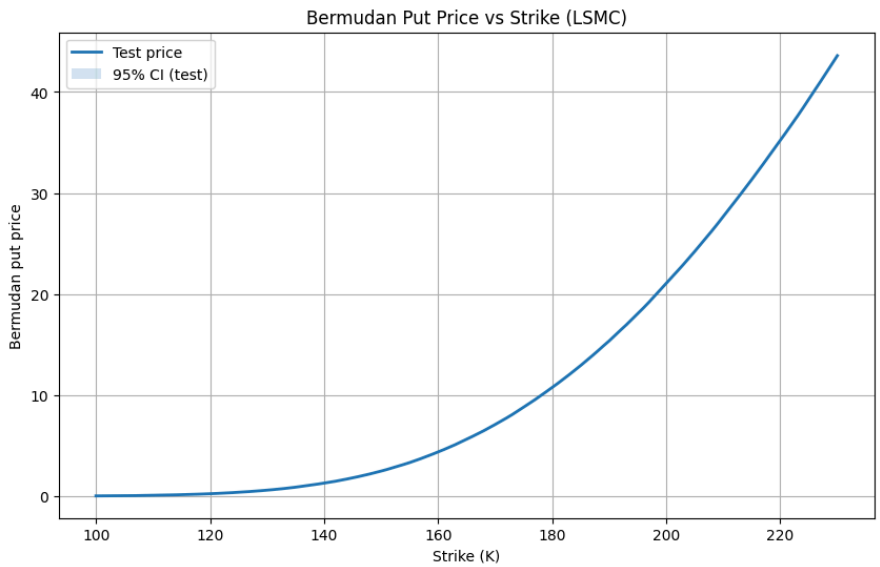
Train / test paths: 20000 / 100000

Seed: Same seed across all strike prices

Strike Price	Market Mid-Quote for Option Premium	Output Lower-Bound Price (LSM)	Absolute Error	% Error	Volume Traded
100	0.615	0.0186	**-0.5964**	**-96.98%**	626
105	0.755	0.0373	**-0.7177**	**-95.06%**	3
110	0.92	0.0723	**-0.8477**	**-92.14%**	0
115	1.115	0.1303	**-0.9847**	**-88.31%**	299
120	1.345	0.2261	**-1.1189**	**-83.19%**	141
125	1.63	0.3733	**-1.2567**	**-77.10%**	22
130	1.965	0.5839	**-1.3811**	**-70.29%**	112
135	2.355	0.8845	**-1.4705**	**-62.44%**	95
140	2.82	1.3054	**-1.5146**	**-53.71%**	421
145	3.35	1.8377	**-1.5123**	**-45.14%**	31
150	4	2.5204	**-1.4796**	**-36.99%**	74
155	4.75	3.3611	**-1.3889**	**-29.24%**	19
160	5.625	4.4148	**-1.2102**	**-21.51%**	172
165	6.65	5.6626	**-0.9874**	**-14.85%**	72

170	7.825	7.1513	**-0.6737**	**-8.61%**	667
175	9.175	8.8494	**-0.3256**	**-3.55%**	105
180	10.675	10.8010	**+0.1260**	**1.18%**	1357
185	12.4	13.0349	**+0.6349**	**5.12%**	1059
190	14.4	15.4773	**+1.0773**	**7.48%**	320
195	16.6	18.1692	**+1.5692**	**9.45%**	170
200	19.125	21.1122	**+1.9872**	**10.39%**	819
205	21.95	24.2513	**+2.3013**	**10.48%**	164
210	24.925	27.7112	**+2.7862**	**11.18%**	1
215	28.6	31.3878	**+2.7878**	**9.75%**	0
220	32.475	35.2820	**+2.8070**	**8.64%**	2
225	36.675	39.3578	**+2.6828**	**7.32%**	5
230	40.95	43.6397	**+2.6897**	**6.57%**	4

This model works best at ATM Options.



#	Market Price	LSM Price	Absolute Error	% Error
1	0.615	0.0186	-0.5964	-96.9756%
2	0.755	0.0373	-0.7177	-95.0596%
3	0.920	0.0723	-0.8477	-92.1413%
4	1.115	0.1303	-0.9847	-88.3139%
5	1.345	0.2261	-1.1189	-83.1896%
6	1.630	0.3733	-1.2567	-77.0982%
7	1.965	0.5839	-1.3811	-70.2850%
8	2.355	0.8845	-1.4705	-62.4416%
9	2.820	1.3054	-1.5146	-53.7092%
10	3.350	1.8377	-1.5123	-45.1433%
11	4.000	2.5204	-1.4796	-36.9900%
12	4.750	3.3611	-1.3889	-29.2400%
13	5.625	4.4148	-1.2102	-21.5147%
14	6.650	5.6626	-0.9874	-14.8481%
15	7.825	7.1513	-0.6737	-8.6096%
16	9.175	8.8494	-0.3256	-3.5488%
17	10.675	10.8010	+0.1260	+1.1803%
18	12.400	13.0349	+0.6349	+5.1202%
19	14.400	15.4773	+1.0773	+7.4813%
20	16.600	18.1692	+1.5692	+9.4530%
21	19.125	21.1122	+1.9872	+10.3906%
22	21.950	24.2513	+2.3013	+10.4843%
23	24.925	27.7112	+2.7862	+11.1783%
24	28.600	31.3878	+2.7878	+9.7476%
25	32.475	35.2820	+2.8070	+8.6436%
26	36.675	39.3578	+2.6828	+7.3151%
27	40.950	43.6397	+2.6897	+6.5683%

Model Accuracy Tests

Brain dump steps

1. Determine optimal J and optimal H first by plotting MSE and train / test price graphs using the same option reference price under different parameters.

2. Run 15 Monte Carlo simulations, take average of test sample prices then compare against [X] amount of WRDS option mid quote data.